

BOOSTING TUNING · HANDS-ON

Bayesian Optimization 으로 부스팅 튜닝하기 (R · Python)

실제 데이터 (은행 마케팅) 에서 Grid · Random · Bayesian 탐색을 같은 조건으로 비교하고 , 최적 튜닝 파라미터를 찾는 전 과정

Python: xgboost + Optuna(TPE) · R: xgboost + ParBayesianOptimization(GP), tidymodels tune_bayes()

질문 : Grid 보다 적은 비용으로 좋은 파라미터를 찾을 수 있나 ?

243 → 60

Python: Grid 243 회 최고 CV AUC
0.7601 ↔ TPE 60 회 평균 최고 0.7617
(3 번 반복)

243 → 40

R: Grid 243 회 최고 0.7597 ↔ GP 기반
BO 40 회 0.7586, 0.7592 (Grid 243
개 중 상위 4.2 위 수준)

≈ 1/4 비용

모형 적합 (5-fold CV) 횟수 기준 .
시간으로도 Python 은 275 초 → 약 50
초

답 : “ 대체로 그렇다 , 단 조건이 있다 ”

① 비교 기준은 “ 모형 적합 횟수 ” . 한 번의 적합이 비쌀수록 (큰 데이터 , 많은 트리) 이득이 커진다 ② 적합이 1~2 초로 싸면 BO 의 계산 부담 (대리모형 적합) 이 오히려 눈에 띈다 (R 사례) ③ 상위권 설정들 사이의 CV AUC 차이 (0.001~0.003) 는 CV 잡음 범위 안 → “ 비슷하게 좋은 설정을 훨씬 적은 시도로 찾는다 ” 가 정확한 표현

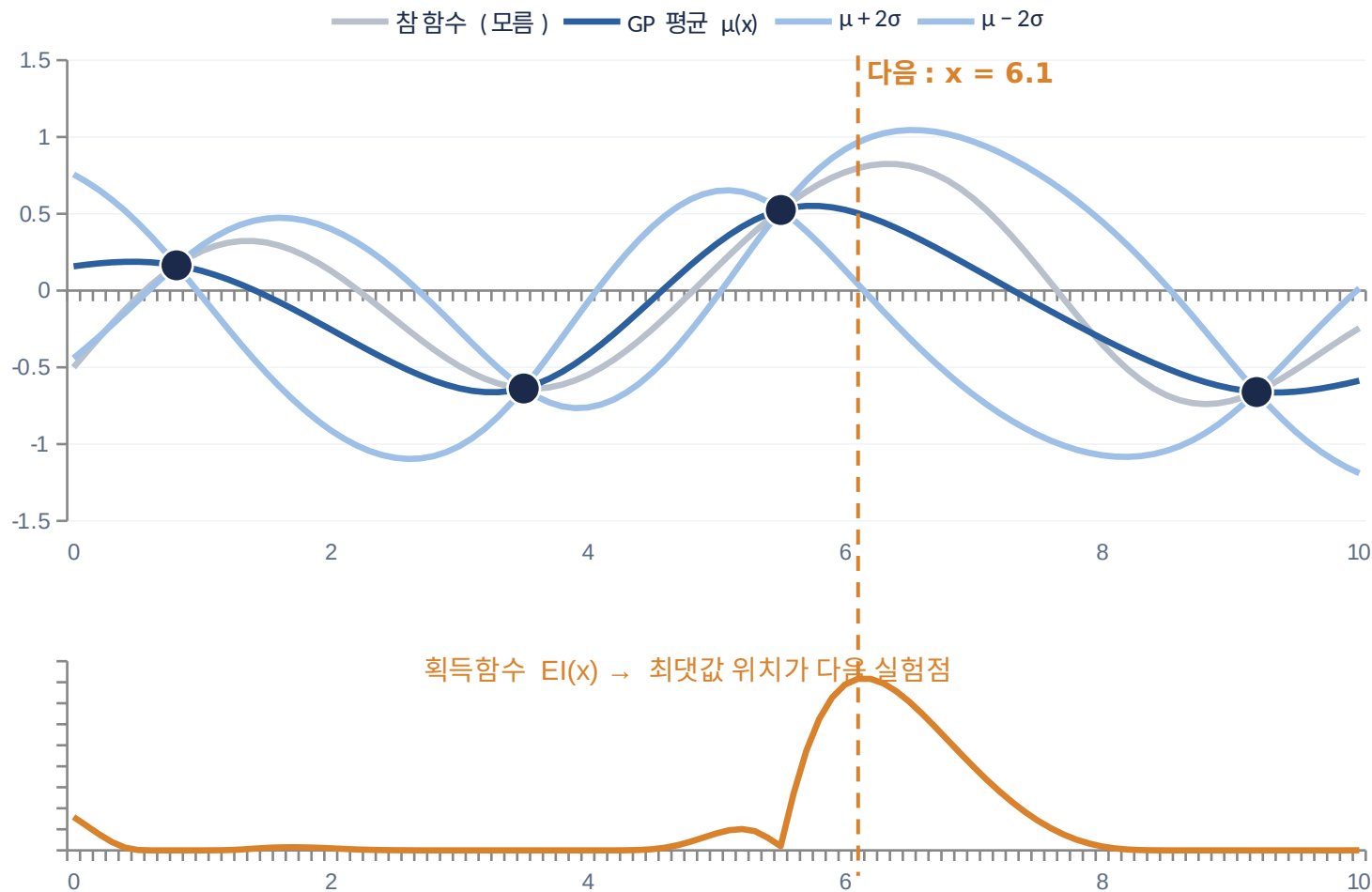
모든 수치는 이 슬라이드를 만들며 직접 실행한 결과 (CPU 2 코어). 자세한 설정은 뒤 슬라이드

반응표면법 (RSM) 과 무엇이 같고 무엇이 다른가

	반응표면법 (RSM)	Bayesian Optimization
반응 y	공정 수율 등 실험 결과	CV 오차 / CV AUC (한 번 = 모형 적합 k 번)
요인 x	온도 , 압력 ...	max_depth, min_child_weight, λ ...
반응표면 모형	1 차 · 2 차 다항 회귀 (국소 근사)	Gaussian process(kriging) 또는 TPE (전역 · 비모수)
불확실성	회귀계수의 표준오차	예측분산 $\sigma^2(x)$: 안 가본 곳일수록 큼
다음 실험점	최대경사 경로 , 정상점 분석	획득함수 EI(x) 를 최대로 하는 점
초기 설계	요인 · 중심합성 설계	무작위 · LHS 5~10 개
강점	해석 가능한 2 차 모형	울퉁불퉁한 표면 , 정수 · 범주형 요인 , 잡음

한 문장으로 : BO = 비모수 반응표면 (GP) 으로 하는 순차적 실험계획 . 반응 하나 얻는 데 드는 비용이 큰 실험을 최소 횟수로 최적화

1 차원 예 : 관측 4 개로 적합한 GP 대리모형



● 관측점 4 개

지금까지 “실험” 한 점 . 최고값 0.53

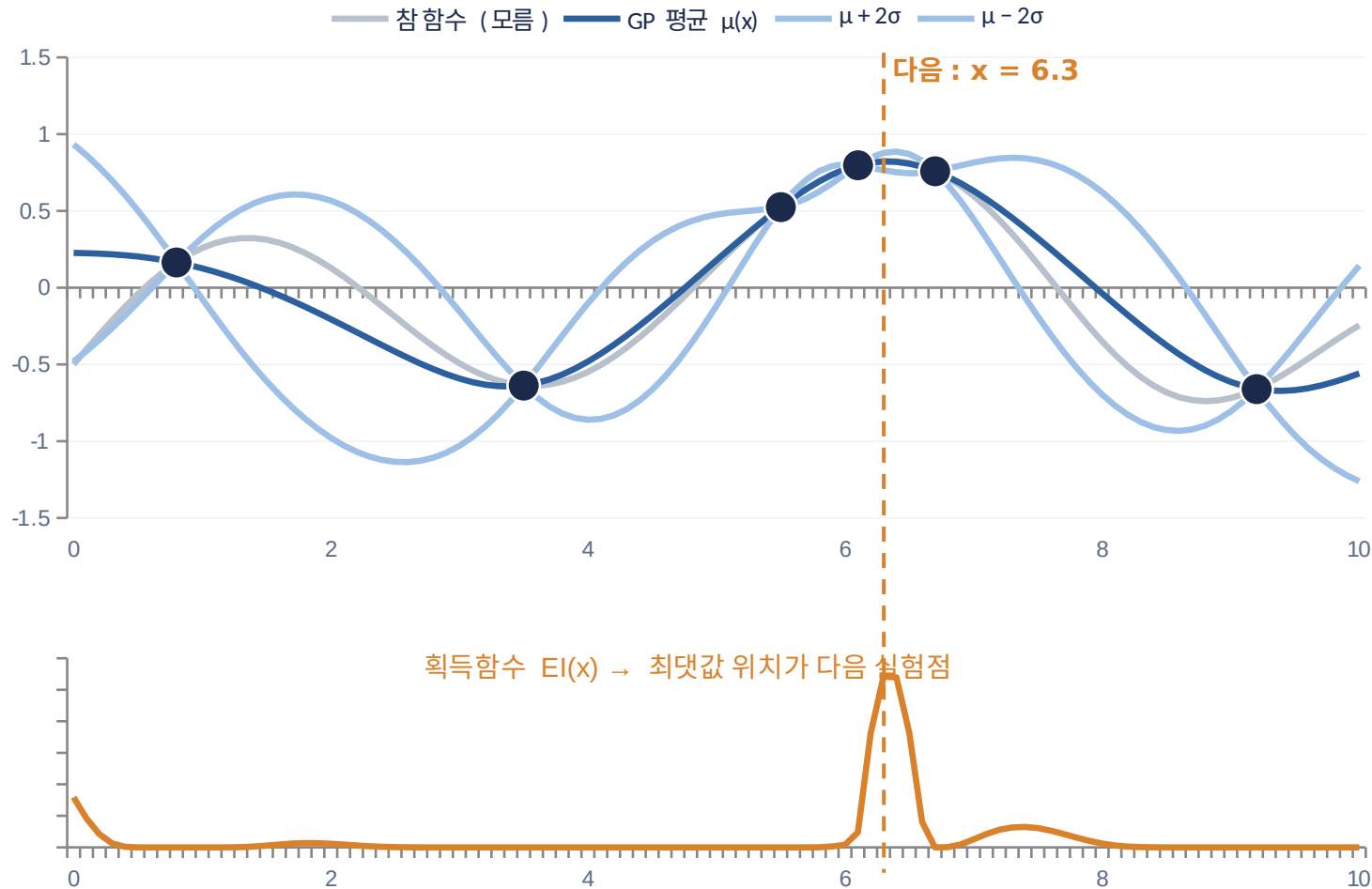
파란 띠 = 불확실성

관측점에서는 좁고 , 멀어질수록 넓다 . 5~7 사이는 아직 모른다

EI 가 고른 다음 점

평균이 높고 (좋을 것 같고) 불확실성도 큰 곳의 균형 → x = 6.1

두 번 더 실험한 뒤 : 최적점 근처로 빠르게 수렴



관측 6 개

최고값 0.53 $\rightarrow$ 0.80. EI 가 고른 다음 점 $x = 6.3$ 은 참 최대 (0.82) 의 위치

불확실성 감소

탐색한 구간의 띠가 좁아지고, 먼 곳은 EI 가 0 에 가까움 $\rightarrow$ 더 볼 필요 없음

Grid 였다면 ?

0.5 간격 격자로는 21 번 실험 . BO 는 7 번째 실험에서 참 최대에 도달

획득함수 (EI) 와 또 다른 대리모형 TPE

Expected Improvement (최대화 문제)

$$EI(x) = (\mu - f^*)\Phi(z) + \sigma\phi(z), \quad z = (\mu - f^*)/\sigma$$

- 첫 항 : 평균이 현재 최고 f^* 보다 높을수록 $\uparrow$ (exploitation)
- 둘째 항 : 불확실성 σ 가 클수록 $\uparrow$ (exploration)
- 대안 : UCB = $\mu + \kappa\sigma$ (κ 가 탐색 강도), PI = $P(f > f^*)$

TPE (Tree-structured Parzen Estimator)

$f(x)$ 를 직접 모형화하지 않고 뒤집어서 본다

시도들을 결과 기준 상위 $\gamma\%$ (좋음) 와 나머지 (나쁨) 로 나눠 , 각 그룹의 x 분포 $l(x)$, $g(x)$ 를 커널밀도로 추정

다음 후보 : $l(x) / g(x)$ 가 큰 x (EI 최대화와 동치임이 알려져 있음)

정수 · 범주형 · 조건부 파라미터 , 높은 차원에 강하고 빠름 $\rightarrow$ Optuna 의 기본값

GP: Snoek, Larochelle & Adams (2012). TPE: Bergstra et al. (2011). 이번 실습: Python = TPE, R = GP (ParBayesianOptimization, tune_bayes)

실습 데이터 : 은행 텔레마케팅 (bank.csv)

4,521

고객 수 (행)

15

설명변수 (수치 6 + 범주 9)

11.5%

정기예금 가입 (y = yes)
비율

AUC

평가지표 (불균형 분류)

duration 제외

통화 시간은 통화가 끝나야 알 수 있는 정보 → 예측 시점에 없는 변수 (누수).
UCI 설명서도 제외를 권장

분할

학습 80% / 테스트 20% (층화). 튜닝은 학습 데이터의 5-fold CV AUC
로만

모형

XGBoost, 학습률 0.05 고정 , 트리 수는 조기 종료 (50 회) 로 결정 →
탐색에서 제외

공정한 비교

세 방법 모두 같은 fold, 같은 탐색 공간 , 같은 목적함수

탐색 공간과 비교 설계

하이퍼파라미터	범위	척도	Grid 수준 (3 개씩)
max_depth	2 ~ 8	정수	2, 4, 6
min_child_weight	1 ~ 50	로그	1, 7, 50
subsample	0.5 ~ 1.0	선형	0.5, 0.75, 1
colsample_bytree	0.5 ~ 1.0	선형	0.5, 0.75, 1
lambda (L2)	0.01 ~ 100	로그	0.01, 1, 100

Grid search $3^5 = 243$ 회**Random search**

60 회 (Python) · 40 회 (R)

Bayesian (TPE)

60 회 , 처음 10 회 무작위 (Python)

Bayesian (GP)

40 회 = 초기 10 + 반복 30 (R)

비용의 단위 = “1 회 = 5-fold CV 한 번 (모형 5 번 적합)”. Grid 는 차원이 하나 늘 때마다 3 배 ($3^6 = 729$, $3^7 = 2,187$) 로 폭발하지만 , Random·BO 는 예산을 우리가 정한다 . Random·TPE 는 시드를 바꿔 3 번 (R 은 2 번) 반복

Grid 수준을 4 개로 늘리면 $4^5 = 1,024$ 회

PART 1

Python 실습

xgboost + Optuna (TPE)

데이터 준비와 목적함수 : “ 실험 한 번 ” 정의하기

PYTHON

```
import pandas as pd, xgboost as xgb, optuna
from sklearn.model_selection import train_test_split, StratifiedKFold

bank = pd.read_csv("bank.csv", sep=";").drop(columns=["duration"]) # 누수 변수 제거
y = (bank.pop("y") == "yes").astype(int)
X = pd.get_dummies(bank).astype(float) # 범주형 → 더미
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, stratify=y, random_state=2026)
dtr = xgb.DMatrix(Xtr, label=ytr)
folds = list(StratifiedKFold(5, shuffle=True, random_state=1).split(Xtr, ytr))

def objective(trial):
    # 입력 : 설정 하나 → 출력 : CV AUC
    p = {"objective": "binary:logistic", "eval_metric": "auc", "eta": 0.05,
         "max_depth": trial.suggest_int("max_depth", 2, 8),
         "min_child_weight": trial.suggest_float("min_child_weight", 1, 50, log=True),
         "subsample": trial.suggest_float("subsample", 0.5, 1.0),
         "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0),
         "lambda": trial.suggest_float("lambda", 1e-2, 100, log=True)}
    cv = xgb.cv(p, dtr, num_boost_round=2000, folds=folds, early_stopping_rounds=50)
    trial.set_user_attr("n_trees", len(cv)) # 조기 종료가 고른 트리 수 저장
    return cv["test-auc-mean"].iloc[-1]
```

탐색 실행 , 결과 확인 , 최종 모형

PYTHON

```
sampler = optuna.samplers.TPESampler(n_startup_trials=10, seed=0) # 처음 10회는
무작위
study = optuna.create_study(direction="maximize", sampler=sampler)
study.optimize(objective, n_trials=60)

print(study.best_value)          # 최고 CV AUC
print(study.best_params)         # 최적 튜닝 파라미터
optuna.visualization.plot_optimization_history(study)
optuna.visualization.plot_param_importances(study)

# 비교 : sampler=optuna.samplers.RandomSampler(seed=0) → Random search

# 최종 모형 : 최적 설정 + 그때의 트리 수로 학습 데이터 전체 적합 → test 평가
from sklearn.metrics import roc_auc_score
bp = study.best_params | {"objective": "binary:logistic", "eta": 0.05}
n = study.best_trial.user_attrs["n_trees"]
model = xgb.train(bp, dtr, num_boost_round=n)
roc_auc_score(yte, model.predict(xgb.DMatrix(Xte)))
```

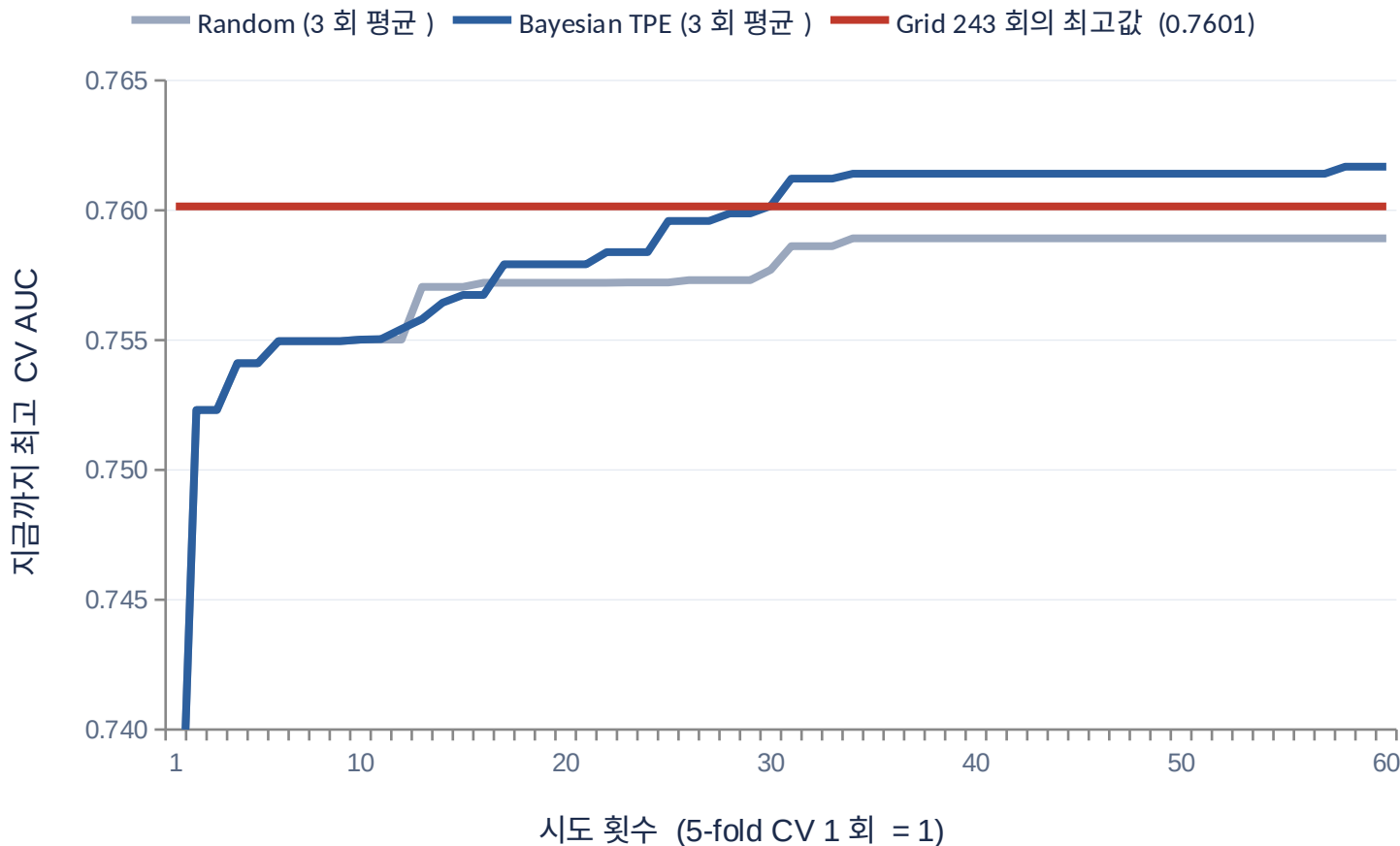
찾은 설정 (seed 0)

```
max_depth = 2
min_child_weight = 1.14
subsample = 0.56
colsample_bytree = 0.53
lambda = 0.67
트리 수 = 281
```

CV AUC = 0.7610

얕은 트리 (깊이 2), 작은 잎 제약, 행·열
부표본 50~60% — 신호가 약한 데이터에서
전형적인 “단순한 모형” 선택

결과 ① : 시도 횟수에 따른 “지금까지의 최고 CV AUC”



처음 10 회

TPE 도 무작위로 시작하므로 두 곡선이 겹친다

그 이후

TPE 는 좋은 영역에 집중 . Grid 최고값 - 0.001 이내 도달 : TPE 31, 17, 15 번째 / Random 못함 , 못함 , 31 번째

60 회 뒤

TPE 최고 0.7610, 0.7628, 0.7612 — 3 번 모두 Grid 243 회 (0.7601) 보다 높음 . Random: 0.7580, 0.7585, 0.7603

y 축은 0.74 부터 시작 (차이를 보기 위해 확대). 한 번의 최고값 선택에는 CV 잡음이 섞여 있음에 유의

결과 ② : 비용과 성능 요약

방법	시도 수	시간 (초)	최고 CV AUC	테스트 AUC
기본값 (튜닝 없음)	0	—	—	0.734
Grid search	243	275	0.7601	0.738
Random search	60	52 (평균)	0.7589 (평균)	—
Bayesian (TPE)	60	50 (평균)	0.7617 (평균)	0.733 (seed 0)

비용

TPE 는 Grid 의 약 ¼ 시도 (60 vs 243), 시간도 약 ½ 수준으로 더 높은 CV AUC 도달

테스트 AUC 해석

테스트 AUC 는 0.734~0.738 로 방법 간 차이가 작다 . 테스트 양성 104 명뿐이라 AUC 의 표준오차가 약 0.025 → 차이를 구분할 수 없음

이 데이터의 교훈

신호가 약한 데이터에서는 튜닝의 이득 자체가 작다 . 그래도 “적은 비용으로 최상위권 설정” 이라는 BO 의 장점은 그대로

PART 2

R 실습

xgboost + ParBayesianOptimization (Gaussian process) · tidymodels tune_bayes()

데이터 준비와 scoring function

R CONSOLE

```
library(xgboost); library(ParBayesianOptimization)
bank <- read.csv("bank.csv", sep = ";", stringsAsFactors = TRUE)
bank$duration <- NULL # 누수 변수 제거
y <- as.integer(bank$y == "yes")
X <- model.matrix(~ . - y, data = bank)[, -1] # 범주형 → 더미
set.seed(2026) # 층화 80/20 분할
te <- c(sample(which(y == 1), round(.2 * sum(y == 1))), sample(which(y == 0), round(.2 * sum(y == 0))))
dtr <- xgb.DMatrix(X[-te, ], label = y[-te])

scoringFunction <- function(max_depth, log_mcw, subsample, colsample_bytree, log_lambda) {
  p <- list(objective = "binary:logistic", eval_metric = "auc", eta = 0.05,
            max_depth = max_depth, min_child_weight = 10^log_mcw,
            subsample = subsample, colsample_bytree = colsample_bytree, lambda = 10^log_lambda)
  cv <- xgb.cv(params = p, data = dtr, nrounds = 2000, folds = folds, # folds: 층화 5-fold 인덱스 목록
              early_stopping_rounds = 50, verbose = 0)
  list(Score = max(cv$evaluation_log$test_auc_mean), # 최대화할 값
       nrounds = cv$early_stop$best_iteration) # 함께 기록할 값
}
```

bayesOpt() 실행과 최적 파라미터

R CONSOLE

```
bounds <- list(max_depth = c(2L, 8L), log_mcw = c(0, log10(50)),
              subsample = c(0.5, 1), colsample_bytree = c(0.5, 1),
              log_lambda = c(-2, 2)) # 정수 경계 (2L, 8L) → 정수로
```

탐색

```
set.seed(1)
opt <- bayesOpt(FUN = scoringFunction, bounds = bounds,
               initPoints = 10, # 초기 설계 : 무작위 (LHS) 10 점
               iters.n = 30, # 이후 GP + 획득함수로 30 점
               acq = "ei") # Expected Improvement
```

```
opt$scoreSummary # 매 시도의 설정 · Score · nrounds
getBestPars(opt) # 최적 튜닝 파라미터
plot(opt) # 시도별 Score 와 획득함수 값
```

```
# 최종 모형 : 최적 설정 + nrounds 로 학습 데이터 전체 적합 → test AUC
```

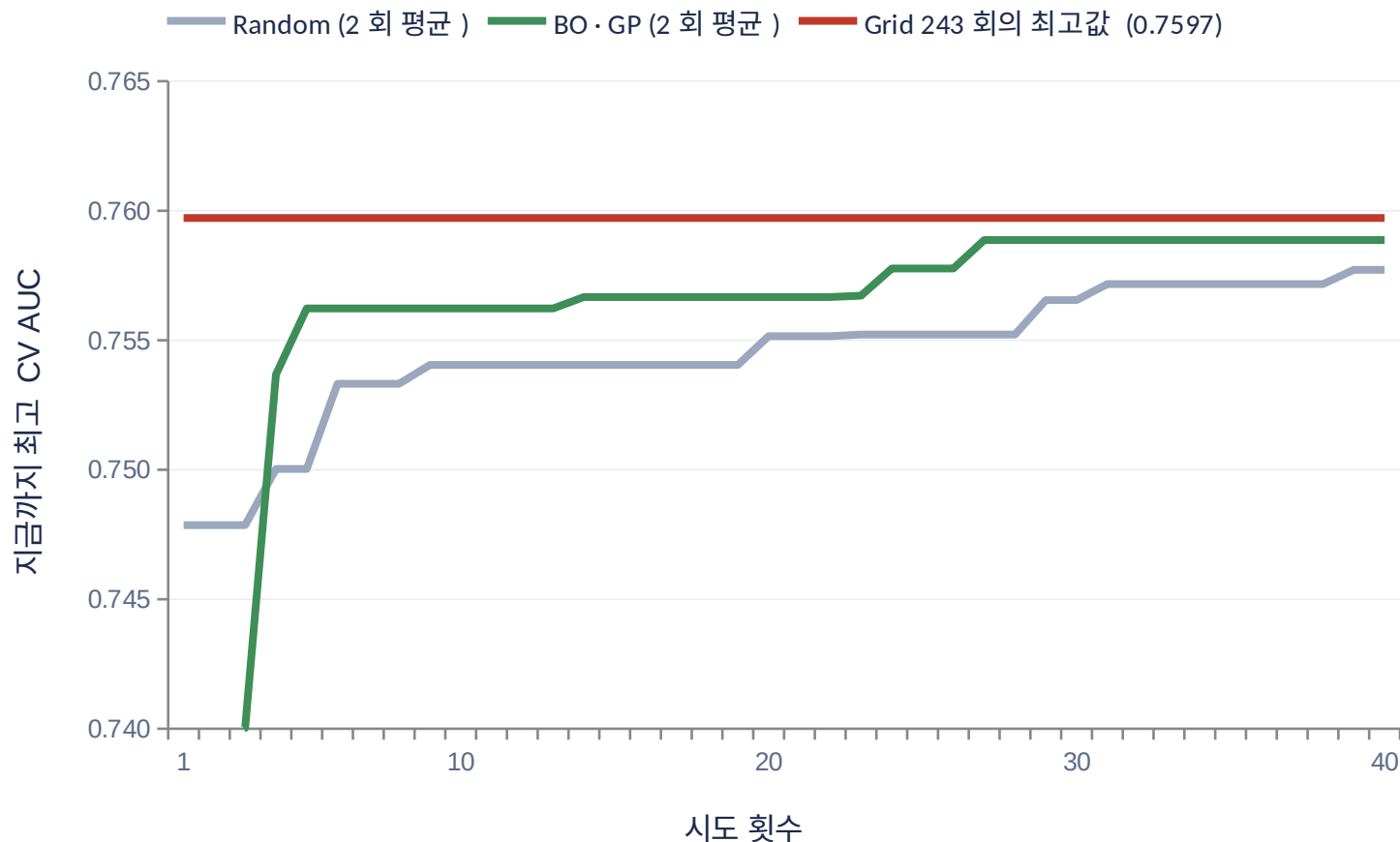
찾은 설정 (seed 2)

```
max_depth = 4
min_child_weight = 1.19
subsample = 0.63
colsample_bytree = 0.50
lambda = 0.27
```

CV AUC = 0.7592

seed 1 은 0.7586. Python 과 같은 방향 : 행 · 열
부표본을 낮게, 앞 제약은 작게

결과 : GP 기반 BO 40 회 vs Random 40 회 vs Grid 243 회



성능

BO 40 회 : 0.7586, 0.7592 → Grid 243 개 중 상위 4 위 · 2 위 수준 . Random 40 회 : 0.7576, 0.7578

시간 (정직한 결과)

Grid 243 회 390 초 , Random 40 회 55 초 , BO 40 회 346 초 . 적합 1 회가 약 1.6 초로 싸서 GP 적합 · EI 최적화 부담이 더 컸다

교훈

모형 한 번이 수 분 걸리는 큰 데이터라면 BO 의 추가 부담은 무시할 만하고 , 적합 횟수 절약이 그대로 시간 절약이 된다

ParBayesianOptimization 1.2.6 (DiceKriging GP), xgboost 3.2 (R), CPU 2 코어 . y 축은 0.74 부터

같은 일을 tidymodels 로 : tune_bayes()

R CONSOLE

```
library(tidymodels) # parsnip, tune, rsample, dials ...
bank$y <- factor(bank$y, levels = c("yes", "no")) # 첫 수준 = 관심 사건
set.seed(2026); sp <- initial_split(bank, prop = 0.8, strata = y)
tr <- training(sp); folds <- vfold_cv(tr, v = 5, strata = y)
spec <- boost_tree(trees = 2000, learn_rate = 0.05, stop_iter = 50,
  tree_depth = tune(), min_n = tune(), sample_size = tune(),
  mtry = tune(), loss_reduction = tune()) |>
  set_engine("xgboost", validation = 0.2, counts = FALSE) |> set_mode("classification")
wf <- workflow(recipe(y ~ ., tr) |> step_dummy(all_nominal_predictors()), spec)
prm <- extract_parameter_set_dials(wf) |>
  update(tree_depth = tree_depth(c(2L, 8L)), min_n = min_n(c(2L, 50L)),
    sample_size = sample_prop(c(.5, 1)), mtry = mtry_prop(c(.5, 1)))
res <- tune_bayes(wf, resamples = folds, param_info = prm, initial = 10, iter = 30,
  metrics = metric_set(roc_auc))
show_best(res, metric = "roc_auc"); autoplot(res, type = "performance")
final <- last_fit(finalize_workflow(wf, select_best(res, metric = "roc_auc")), sp)
collect_metrics(final) # test AUC
```

실행 결과

40 회 , 101 초 . 최고 CV AUC 0.7569,
테스트 AUC 0.734

차이점

GP + TI가 기본 . 트리 수는 내부 검증 20%
로 조기 종료 . 분할이 달라 앞 결과와 수치는
직접 비교 불가

장점

전처리 (recipe) 까지 CV 안에서 → 누수
방지 . 엔진만 바꾸면 lightgbm(bonsai) 도
동일

언제 Bayesian optimization 이 확실히 이득인가

유리한 경우

- 모형 한 번 적합이 비쌀 때 (큰 데이터 , 많은 트리 , 딥러닝)
- 하이퍼파라미터가 4~10 개 정도일 때 (Grid 가 폭발하는 영역)
- 예산이 수십 회로 제한될 때

이득이 작은 경우

- 적합이 1~2 초로 매우 쌀 때 → Random 을 많이 돌리는 게 더 간단
- 시도를 수백 회 이상 할 수 있을 때 → Random 과의 차이 감소
- 병렬 자원이 많을 때 (Grid·Random 은 완전 병렬)

주의

- CV AUC 0.001~0.003 차이는 잡음 → 상위권 중 단순한 설정 선택
- CV 에 과적합 가능 → 최종 확인은 test 또는 nested CV
- 탐색 공간 (범위 · 로그 척도) 설계가 결과를 좌우

실무 권장 : 학습률 고정 + 조기 종료 → 4~6 개 파라미터를 BO 30~60 회 → 학습률 낮춰 재적합

더 빠르게 : pruning 과 병렬 실행

Python · Optuna

PYTHON

```
study = optuna.create_study(
    direction="maximize",
    sampler=optuna.samplers.TPESampler(seed=0),
    pruner=optuna.pruners.MedianPruner())
# 목적함수 안에서 중간 점수 보고 :
# trial.report(score, step)
# if trial.should_prune(): raise optuna.TrialPruned()
study.optimize(objective, n_trials=100, n_jobs=2)
```

가망 없는 시도는 중간에 중단 (pruning) → 체감 시간 수 배 단축

R

- bayesOpt(..., iters.k = 2, parallel = TRUE) + doParallel: 한 번에 여러 점을 병렬 평가
- tune_bayes(): control_bayes(no_improve = 10) 으로 개선이 없으면 조기 중단
- 시간 제한 : bayesOpt(otherHalting = list(timeLimit = 600))
- mlr3mbo: 대리모형 (GP, 랜덤포레스트) 과 획득함수를 자유롭게 선택

공통 : 트리 수는 절대 탐색하지 말고 조기 종료에 맡기기 — 가장 큰 비용 절감

SUMMARY

정리

1 개념

BO = GP/TPE 반응표면 + 획득함수로 다음 실험점을 고르는 순차적 실험계획

2 실험 결과

Python: TPE 60 회가 Grid 243 회보다 높은 CV AUC. R: GP 40 회로 Grid 243 개 중 최상위권

3 비용

적합 횟수는 $\frac{1}{4} \sim \frac{1}{6}$. 적합이 쌀 땐 BO 의 계산 부담이 보일 수 있음

4 코드

Python: Optuna(TPE) · R: ParBayesianOptimization(GP), tidymodels tune_bayes()